DSA0602-Data Handling And Visulization

Class Activity 2

Name: Nanabala Harshitha

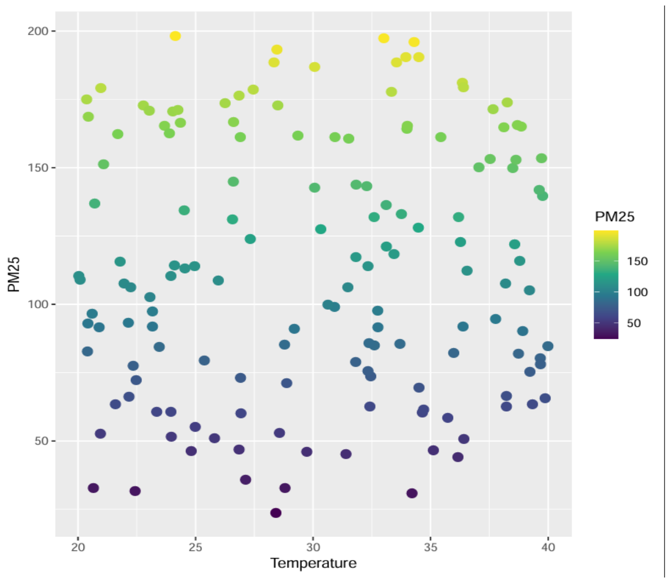
Reg No.:192324261

1. **Smart City Air Quality Monitoring:**

   Python code:

```
air<- ata.frame(
  Location=paste("L", 1:150, sep=""),
  PM25=runif(150, 20, 200),
  NO2=runif(150, 10, 120),
  CO=runif(150, 0.5, 8),
  Temperature=runif(150, 20, 40),
  Humidity=runif(150, 30, 90)
)
ggplot(air, aes(x=Temperature, y=PM25, colour=PM25)) +
  geom_point(size=3)+scale_colour_viridis_c
```

   Output:



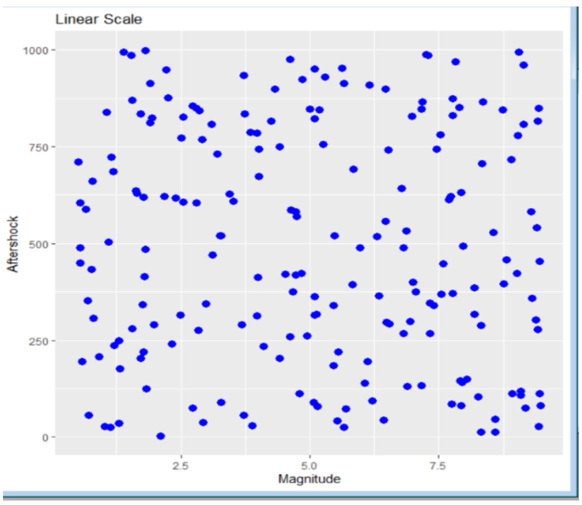
2. **Earthquake Risk Analysis:**

Python code:

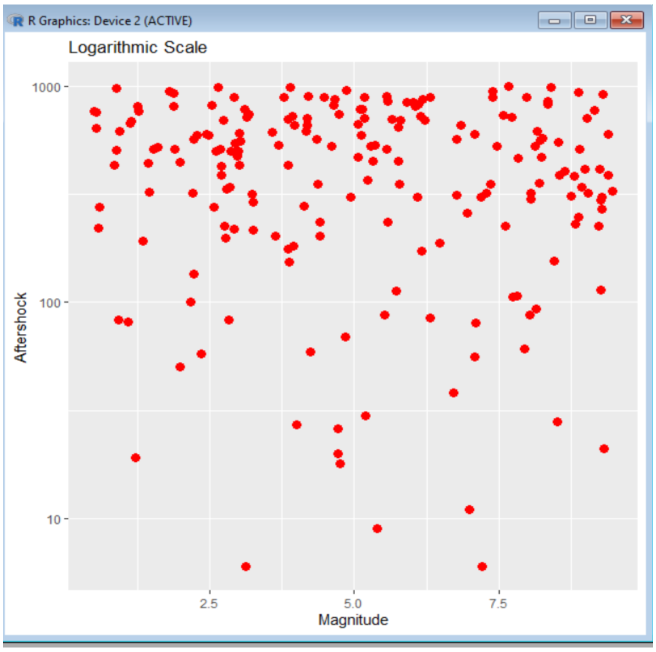
```
earthquake <- data.frame(
  Magnitude = runif(200, 0.5, 9.5),
  Depth = runif(200, 1, 700),
  Latitude = runif(200, -90, 90),
  Longitude = runif(200, -180, 180),
  Aftershock = sample(1:1000, 200, replace = TRUE)
)
ggplot(earthquake, aes(x = Magnitude, y = Aftershock)) +
  geom_point(color = "blue", size = 3) +
  ggtitle("Linear Scale")
ggplot(earthquake, aes(x = Magnitude, y = Aftershock)) +
  geom_point(color = "red", size = 3) +
  scale_y_log10() +
  ggtitle("Logarithmic Scale")
```

Output:

Linear



Non-linear



3. Wind Direction Analysis for Airport Operations:

   Python code:

   Wind<-data.frame(

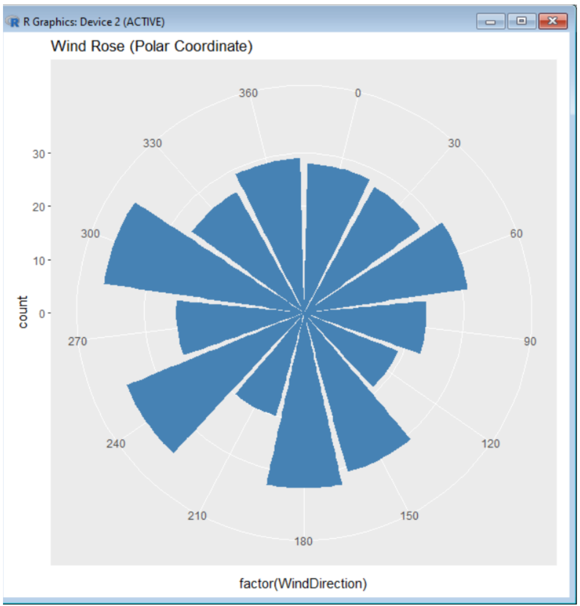
   windspeed=runif(365,5,40),

   windDirection=sample(seq(0,360,by=30),365,replace=TRUE),

   Season=sample(c(“Winter”,”Summer”,”Monsoon”,”Autumn”),365,replace =TRUE))

   gggplot

Output:



4. Hospital Patient Monitoring Dashboard:

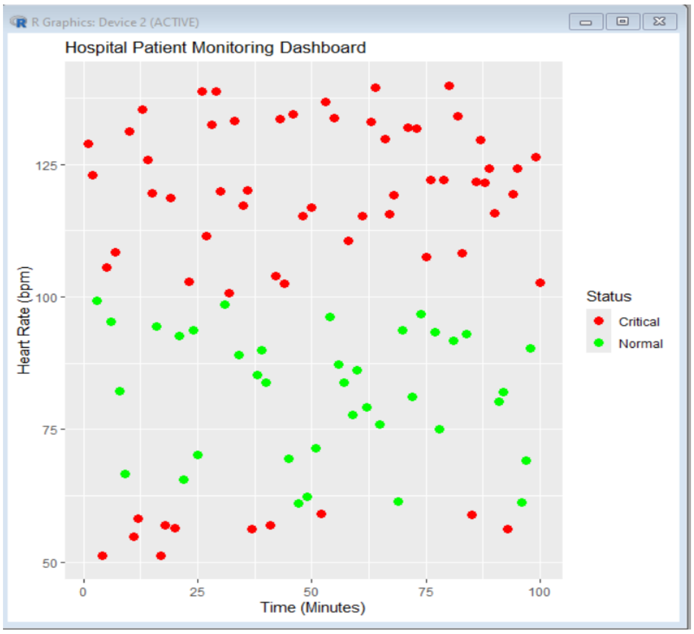
Python code:

```
patient <- data.frame(
  Time = 1:100,
  HeartRate = runif(100, 50, 140),
  Oxygen = runif(100, 80, 100),
  BP = runif(100, 80, 180),
  Respiration = runif(100, 10, 35),
  Temperature = runif(100, 35, 40)
)
patient$Status <- ifelse(patient$HeartRate < 60 | patient$HeartRate >
100,"Critical", "Normal")
ggplot(patient, aes(x = Time, y = HeartRate, color = Status)) +
  geom_point(size = 3) +
  scale_color_manual(values = c("Normal" = "green",
                                "Critical" = "red")) +
  ggtitle("Hospital Patient Monitoring Dashboard") +
```

xlab("Time (Minutes)") +

ylab("Heart Rate (bpm)")

Output:



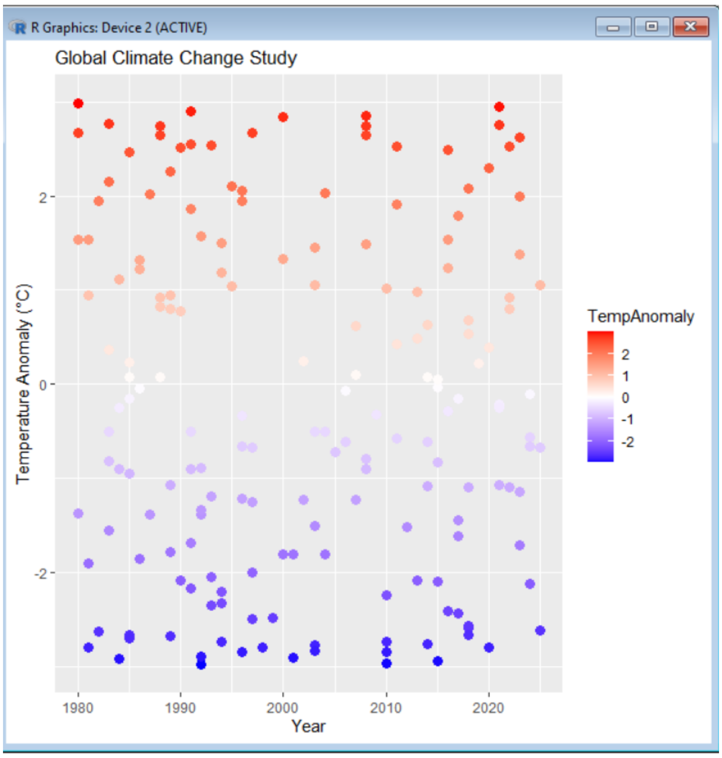
5. Global Climate Change Study:

Python code:

```
climate <- data.frame(
  Country = paste("Country", 1:180),
  Year = sample(1980:2025, 180, replace = TRUE),
  TempAnomaly = runif(180, -3, 3)
)
ggplot(climate, aes(x = Year, y = TempAnomaly, color = TempAnomaly)) +
  geom_point(size = 3) +
  scale_color_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0
  ) +
```

```
  ggtitle("Global Climate Change Study") +
  xlab("Year") +
  ylab("Temperature Anomaly (°C)")
```

Output:



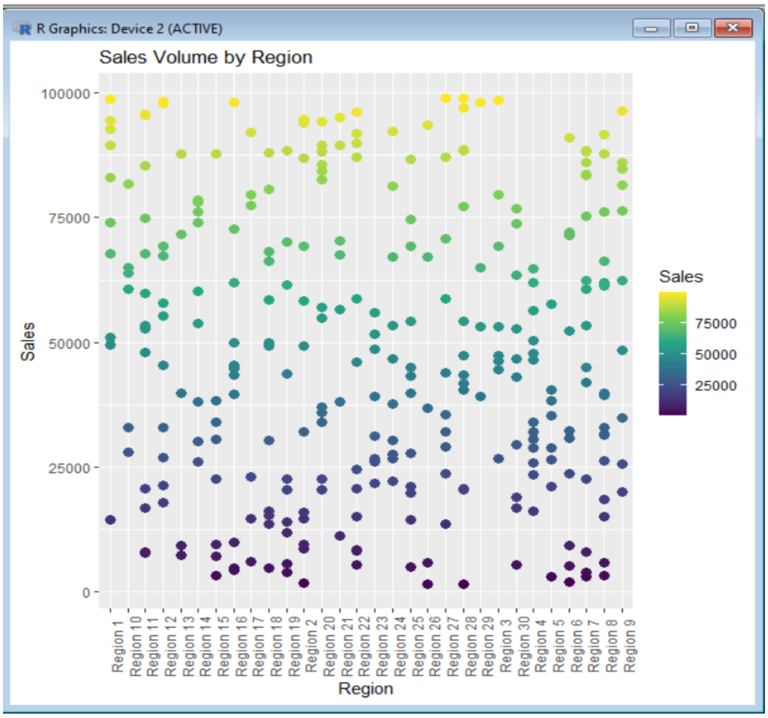
6. E-Commerce Sales Analytics:

Python code:

```
sales <- data.frame(
  Category = sample(paste("Category",1:10), 300, replace = TRUE),
  Region = sample(paste("Region",1:30), 300, replace = TRUE),
  Sales = runif(300,1000,100000)
)
ggplot(sales, aes(x=Region, y=Sales, color=Category)) +
  geom_point(size=3) +
  scale_color_brewer(palette="Set3") +
  ggtitle("Product Categories by Region") +
  theme(axis.text.x = element_text(angle=90))
```

```
ggplot(sales, aes(x=Region, y=Sales, color=Sales)) +
  geom_point(size=3) +
  scale_color_viridis_c() +
  ggtitle("Sales Volume by Region") +
  theme(axis.text.x = element_text(angle=90))
```

Output:



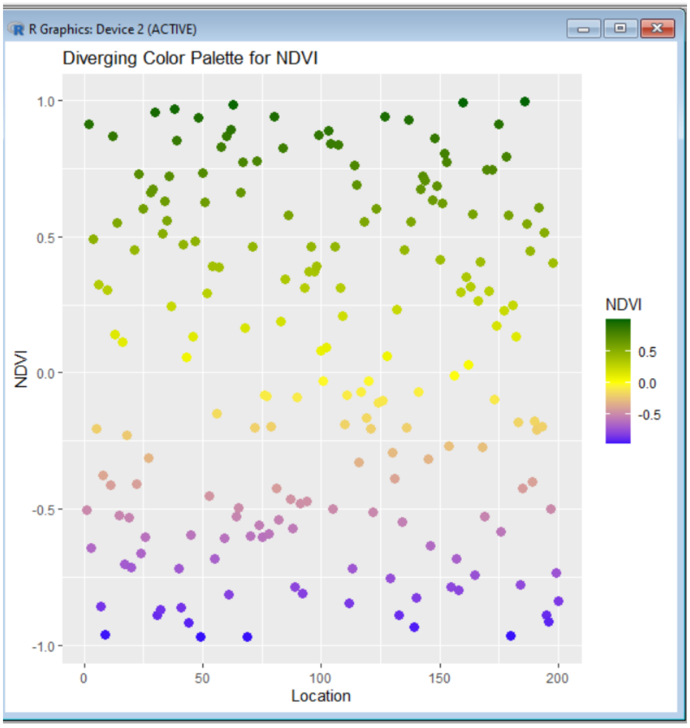
7. Satellite Image Vegetation Analysis:

Python code:

```
ndvi <- data.frame(
  Location = 1:200,
  NDVI = runif(200, -1, 1)
)
ggplot(ndvi, aes(x = Location, y = NDVI, color = NDVI)) +
  geom_point(size = 3) +
  scale_color_viridis_c() +
  ggtitle("Sequential Color Palette for NDVI")
```

```
ggplot(ndvi, aes(x = Location, y = NDVI, color = NDVI)) +
 geom_point(size = 3) +
 scale_color_gradient2(
  low = "blue",
  mid = "yellow",
  high = "darkgreen",
  midpoint = 0
 ) +
 ggtitle("Diverging Color Palette for NDVI")
```

Output:



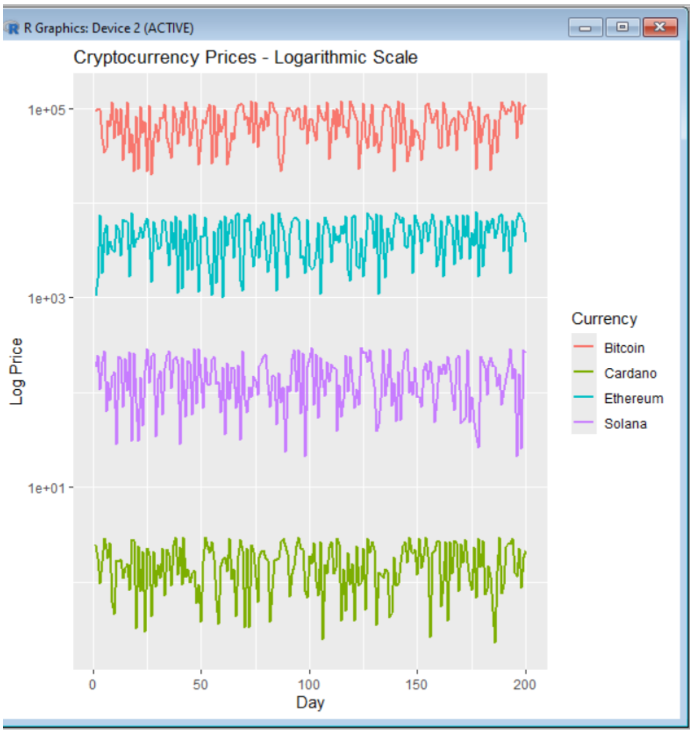
8. Cryptocurrency Market Volatility:

Python code:

```
crypto <- data.frame(
 Day = 1:200,
 Bitcoin = runif(200, 20000, 120000),
```

```
  Ethereum = runif(200, 1000, 8000),
  Solana = runif(200, 20, 300),
  Cardano = runif(200, 0.2, 3)
)
crypto_long <- data.frame(
  Day = rep(crypto$Day, 4),
  Price = c(crypto$Bitcoin, crypto$Ethereum,
          crypto$Solana, crypto$Cardano),
  Currency = factor(rep(c("Bitcoin",
                         "Ethereum",
                         "Solana",
                         "Cardano"),
                       each = 200))
)
ggplot(crypto_long,
      aes(x = Day, y = Price, color = Currency)) +
  geom_line(size = 1) +
  ggtitle("Cryptocurrency Prices - Linear Scale") +
  xlab("Day") +
  ylab("Price")
ggplot(crypto_long,
      aes(x = Day, y = Price, color = Currency)) +
  geom_line(size = 1) +
  scale_y_log10() +
  ggtitle("Cryptocurrency Prices - Logarithmic Scale") +
  xlab("Day") +
  ylab("Log Price")
```

Output:



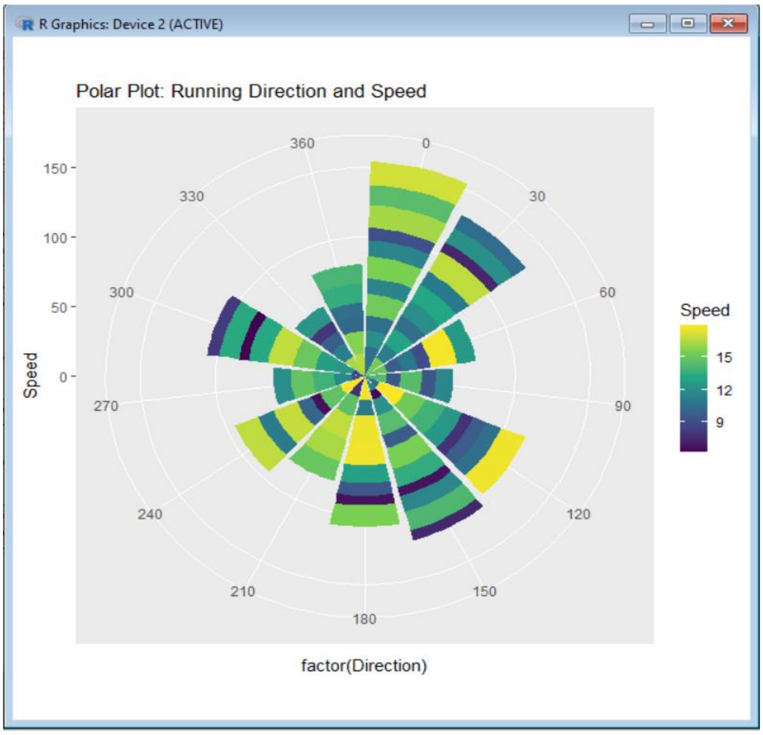
9. Marathon Runner Performance Analysis:

Python code:

```
runner <- data.frame(
  Distance = seq(1, 100),
  Speed = runif(100, 6, 18),
  HeartRate = runif(100, 60, 180),
  Elevation = runif(100, 0, 500),
  Direction = sample(seq(0, 360, by = 30), 100, replace = TRUE)
)
ggplot(runner, aes(x = Distance, y = HeartRate, color = Speed)) +
  geom_point(size = 3) +
  scale_color_viridis_c() +
  ggtitle("Cartesian Plot: Heart Rate vs Distance")
ggplot(runner, aes(x = factor(Direction), y = Speed, fill = Speed)) +
  geom_bar(stat = "identity") +
```

```
  coord_polar() +
  scale_fill_viridis_c() +
  ggtitle("Polar Plot: Running Direction and Speed")
```

Output:



10. COVID-19 Vaccination Campaign Dashboard:

Python code:

```
vaccine <- data.frame(
  District = sample(paste("D",1:20,sep=""), 200, replace = TRUE),
  AgeGroup = sample(c("Children","Adults","Senior"), 200, replace =
TRUE),
  VaccineType = sample(c("Covaxin","Covishield","Sputnik"), 200, replace
= TRUE),
  VaccinationRate = runif(200, 20, 100)
)
```

```
ggplot(vaccine,
    aes(x = District, y = VaccinationRate, color = VaccineType)) +
 geom_point(size = 3) +
 scale_color_brewer(palette = "Set2") +
 ggtitle("Vaccination Dashboard - Vaccine Types") +
 theme(axis.text.x = element_text(angle = 90))
ggplot(vaccine,
    aes(x = District, y = VaccinationRate,
      color = VaccinationRate)) +
 geom_point(size = 3) +
 scale_color_viridis_c() +
 ggtitle("Vaccination Dashboard - Vaccination Rates") +
 theme(axis.text.x = element_text(angle = 90))
```

Output: